

Lecture 01

Contents

1. [Variables](#)
 - Rules for choosing variables
 - Naming conventions
 - Creating variables
 - Deleting variables
 - Checking the types of variables
 - Reading variables
 - Printing variables
2. Some useful functions
 - [dir\(\)](#)
 - [help\(\)](#)
 - [type\(\)](#)
 - [id\(\)](#)
 - [print\(\)](#)
 - [input\(\)](#)
 - [eval](#)
3. [Summary](#)

Variables

A very long time ago, programmers had to remember (addresses) where values are stored in memory in order to access them in code. Now programmers use names (of variables) to access values. These variables can be considered as containers in which values are stored. This is a huge improvement because for humans, a numeric address for a variable is much more difficult to work with than a descriptive identifier.

A variable is a named reference to a value stored in memory. It is like a container that holds data. The data can be (and is often) changed multiple times during program execution. The name of a variable should be descriptive, avoid single letter variables except when used as a iterator variable. Follow the coding standards of your organization.

Variables are locations in memory that can access via a name.

We will talk about scope (where they are accessible) of variables in week 5 when we talk about `functions` and mutability in week 6 when we cover `str`.

Unlike most other languages (JavaScript is a notable exception), python variables does not require explicit type declaration. Their type is inferred from the value assigned to them.

Rules for naming variables and or identifiers

An identifier is just a name for refers to something in your program, it could be the name of a variable, a function, a class or a module.

- Can contain letters, digits, and underscores
- May not start with a digit or contain a space or any python operator (e.g. +, -, *, /, %, =, **, //)
- May not contain a python symbol (e.g. ,, :, ., -, (,), [,], {, }, @, ...)
- Cannot be a Python keyword (e.g. if, else, elif, for, def, class, lambda, while, for)

```
#valid names for variables
variable_1 = 10
_name = 'Narendra'
last_name = 'Pershad'
pi = 3.14
```

```
#invalid names for variables
# first name = 'Narendra'
# last-name = 'Pershad'
# 1st_name = 'Narendra'
```

```
# can you explain why the above variable names are invalid?
```

Styles (conventions) for naming variables in this course

We will cover general coding styles later in the course, but for now we will provide some guidelines on choosing names for variables.

- Use names that are descriptive
- Multiple word should be joined with an underscore _
- Avoid double underscores at the start of an identifier __ except when working with class members. The double underscores are used internally by python for its own identifiers.
- Avoid l (lowercase el) and O (uppercase oh), because on some OS's a user can be easily be confused with the digits 1 and 0.
- It is okay to use single letter variable for simple iterations

```
#prefer the following names
prof_name = 'Narendra Pershad'
source_file = 'variable_names.py'
```

```
#avoid the following names
```

Variables are created when you first assigned a value to it.

All variables must be initialized at declaration

Values are assign using the assignment (=) operator

e.g. `count = 5` # creates a variable called '**count**' and assigns the value 5 to it.

```
# Assigning values to variables
age = 25 # int
```

```
price = 19.99          # float
message = 'Hello!'    # str
is_active = True       # bool

# Reassigning a variable/changed the value of a variable
price = 29.99          # Updated value
```

Variables are case-sensitive

age, Age, AGE are treated as different variables

```
print(age)             # works, because age does exist
print(Age)             # error, because Age does not exist
print(AGE)             # error, because AGE does not exist
```

Checking the type of a variables

Use the `type()` function to check the type of a variable

```
# Checking the type of a variable
print(type(age))       # Output: <class 'int'>
print(type(price))     # Output: <class 'float'>
print(type(message))   # Output: <class 'str'>
print(type(is_active)) # Output: <class 'bool'>
```

Reading variables

You can read the value of a variable by its name.

The python interpreter will go to the variable location, read the value and decode it properly.

Printing the value of a variable

Use the `print()` function to display the value of a variable

We will cover this function in more detail later

```
# Using variables in operations
discount = 0.1
cost = price * (1 - discount)
print(cost)                # Output: 26.991
print(price * (1 - discount)) # also produces the same output

print('You can suppress the automatic line feed', end='') #the next output
continues on this this line
print(', by utilizing the second argument of the print function.')

print(cost, age, message, is_active) # you may display multiple variable
with a single print call
```

Delete or undeclare a variables

Use the `del` operator to remove a variable from the current environment

```
del cost # Deleting a variable
```

```

# Attempting to print a deleted variable will raise an error
# print(cost) # Uncommenting this line will raise a NameError
# variables are references to objects in memory
a = [10]
b = a
print(b) # Output: 10
# Changing the value of 'a' does not affect 'b'
a = a.append(20)
print(b) # Output: 10

```

Some useful Functions

We will study functions later in this course, but for now you may think of it as something that can accomplish a specific task and a way to simplify coding tasks.

We will look at some function, that will be used frequently in this course.

These functions are built-in function in python i.e. they are available without doing any explicit imports.

dir()

The `dir()` function is used to list the names of attributes and methods of an object. It is extremely useful for introspection. It helps you to explore what an object can do, especially in interactive environments like the Python REPL or Jupyter Notebook.

When called without argument it returns a list of names in the current local scope.

```

dir()
# returns a list of names associated with the argument
dir(str) # Output: List of attributes and methods of the str class

```

help()

The `help()` function provides interactive access to Python's documentation (docstrings). It is extremely useful for getting quick information about modules, classes, functions, keywords, and other Python objects.

When called without arguments it enters the interactive help module

```

help()
# displays the documentation for the str class
help(str)

```

A super useful way to browse all of the installed python libraries, is to use the following command:

```
python -m pydoc -b
```

This will create a virtual local web site that will allow you to browse the various attribute and methods from your web browser.

If you want to just browse a particular topic, then the following command will suffice

```
python -m pydoc str|int|dict
```

```
python -m pydoc -n <hostname>
```

will start the server listening at the given hostname

`python -m pydoc -p 1234` will start the server listening at the given port

type()

`type()` returns the datatype (class) of the argument

This function can also be used to dynamically create a class. (Advanced usage)

```
# Checking the type of a variable
print(type(age))      # Output: <class 'int'>
print(type(price))    # Output: <class 'float'>
print(type(message))  # Output: <class 'str'>
print(type(is_active))# Output: <class 'bool'>
print(type(None))     # Output: <class 'NoneType'>

print(type([1, 2, 3])) # Output: <class 'list'>
print(type((1, 2, 3))) # Output: <class 'tuple'>
print(type({'1': 'one'})) # Output: <class 'dict'>
print(type({1, 2, 3}))  # Output: <class 'set'>
```

```
print(type(type(1)))
```

The `issubclass()` and `isinstance()` are also useful to check the type of an object.

id()

In our implementation of python, `id()` corresponds to the memory address of the object

No two objects will have the same `id()`, at the same time (unless they are the same object)

If two variables reference the same object, their `id()` will be the same

```
a = 10
b = a

print(id(a))

print(id(b)) # Should be the same as id(a) since b references the same
object
# the is operator checks if two variables point to the same object
print(a is b) # Output: True
c = 11
c = 10
print(a is c) # Output: True, because c was reassigned to the same value as
a
```

input()

The `input()` function is used to take user input while the program is running.

- It always returns a string even if the user types a number!
- If you want an int or float, then you will have to do the necessary conversion.

- It takes an optional message that is displayed before the input is taken.
- When called, it pauses program execution and waits for the user to type something.

```
name = input('Enter your name: ')
print(name)  # Output: whatever the user inputs

age = int(input('Enter your age: '))
print(age)  # Output: whatever the user inputs, converted to an integer
# Note: If the user inputs a non-integer value, this will raise a ValueError
```

eval()

`eval()` is a powerful but dangerous function - use with utmost caution.

It evaluates strings as python code and returns the result.

It should be avoided at all cost in production code.

However, we will use this function quite frequently in our notebooks because it demonstrates some features of code exquisitely.

```
print(eval('2 + 3'))  # Output: 5, evaluates the expression as Python code
user_input = '{"name": "Alice", "age": 30}'
data = eval(user_input)  # Converts to dict (but prefer `json.loads()`)
print(data)  # Output: {'name': 'Alice', 'age': 30}'
```

Summary

- Variables: Variables are named references to values stored in memory, acting like containers that hold data which can change during program execution.
 - Rules: Variable names (identifiers) must start with a letter or underscore, can include letters, digits, and underscores, but cannot contain spaces, operators, symbols, or be Python keywords.
 - Conventions: Use descriptive names, separate words with underscores, avoid confusing characters like l and O, and reserve double underscores for special class-related use.
 - Case Sensitivity: Python treats variable names as case-sensitive, so age, Age, and AGE are considered different variables.
 - Initialization: Variables are created and initialized when a value is first assigned using the = operator.
- Dynamic Typing: Python does not require explicit type declarations; the type is inferred from the assigned value.

- Reading and Printing: Use the variable name to read its value and `print()` to display it. The interpreter fetches and decodes the value from memory.

Useful Built-in Functions:

- `input()` gets user input (always returns a string).
- `print()` sends text to the screen.
- `dir()` lists attributes/methods of objects.
- `help()` accesses documentation.
- `eval()` executes a string as Python code (use cautiously).
- `type()` to check a variable's data type.
- `id()` to get the memory address of an object.